



Modernise & Migrate: DWER

TravelReporting — App Dev, DevOps & Architects Track

Welcome to **Day 2 (Build & Migrate)** and **Day 3 (Harden & Deliver)** of the Application Modernisation OpenHack for the Department of Water and Environmental Regulation (DWER). Day 1 (Discovery & Orientation) is already complete.

Over the next two days you will transform a 20-year-old ColdFusion travel approval system into a modern, cloud-hosted .NET 8 minimal API backed by Azure SQL, with a choice of React + Vite or Blazor Server frontend, deployed to Azure App Service via Azure DevOps pipelines.

Why this track?

App Developers, DevOps Engineers, and Architects are the engine room of any modernisation programme. This track puts you in the driver's seat: you will analyse legacy code, design a target architecture, migrate frontend and backend, write tests, secure the application, and automate delivery — all with GitHub Copilot as your pair programmer, planner, and agent. By the end you will have a repeatable playbook for modernising government legacy systems with AI assistance.

Your Tools

GitHub Copilot Chat Modes

Mode	When to use	Think of it as...
 Ask	Understanding unfamiliar code, asking "what does this do?", getting explanations, reviewing security	Your senior dev on call

Mode	When to use	Think of it as...
 Plan	Designing architecture, choosing patterns, planning migrations, scaffolding multi-file changes	Your tech lead writing the spec
 Agent	Generating code, creating files, running multi-step tasks end-to-end	Your junior dev who never sleeps

Rule of thumb: Ask → Plan → Agent. Understand before you plan, plan before you generate.

Azure & AI Accelerators

Tool	When it helps
Azure MCP Server	Live Azure resource exploration, deployment validation, KQL log queries, IaC scaffolding — without leaving VS Code
Azure Skills for GitHub Copilot	One-liner commands like <code>@azure-prepare</code> , <code>@azure-deploy</code> , <code>@azure-diagnostics</code> that invoke Azure operations via Copilot
Awesome Copilot	Reusable <code>.prompt.md</code> and <code>.instructions.md</code> files from github.com/github/awesome-copilot — grab pipeline, test, and security prompts rather than writing from scratch
GitHub Advanced Security / CodeQL	Automated OWASP scanning on your migrated .NET code — critical for Phase 7

The Application

Current State

The **TravelReporting** system is a travel approval and reporting application built for DWER (Western Australia) in 2002. Staff use it to propose, approve, and report on official travel.

Dimension	Details
Language / Runtime	Adobe ColdFusion (CFML) — no version pinning found
Frontend	HTML framesets (<code>TravelReporting/Default.cfm</code>), inline CSS, vanilla JavaScript in <code><script></code> blocks
Backend	Monolithic, page-per-action pattern — no service layer, no DI, no ORM
Data access	Raw SQL in <code><cfquery></code> tags throughout (e.g. <code>TravelReporting/Travel/Edit.cfm</code> , <code>TravelReporting/Search/Select.cfm</code>)
Database	Oracle (schema <code>TRRE</code>) with ~10 tables and 4 cross-schema views
Authentication	Windows NT integrated auth via <code>CGI.AUTH_USER</code> (<code>TravelReporting/application.cfm</code>)
CI/CD	None
Tests	None
Key security risk	Unparameterised SQL throughout — SQL injection on every data-entry form

Key source files to reference throughout this hackathon:

File	Purpose
<code>TravelReporting/application.cfm</code>	Application bootstrap, NT auth, session setup
<code>TravelReporting/Default.cfm</code>	Frameset entry point
<code>TravelReporting/Menu.cfm</code>	Navigation — lists all functional areas
<code>TravelReporting/Travel/Edit.cfm</code>	Travel proposal form (~60 fields)

File	Purpose
<code>TravelReporting/Travel/Save.cfm</code>	Save handler with inline SQL
<code>TravelReporting/Travel/View.cfm</code>	View travel record
<code>TravelReporting/Travel/Delete.cfm</code>	Delete handler
<code>TravelReporting/Search/Select.cfm</code>	Search filters
<code>TravelReporting/Search/ReportDisplay.cfm</code>	Search results and report display
<code>TravelReporting/Reports/Monthly/Display.cfm</code>	Monthly report
<code>TravelReporting/Reports/Quarterly/Display.cfm</code>	Quarterly report
<code>TravelReporting/Graphs/Monthly/Display.cfm</code>	Monthly spend graph
<code>TravelReporting/Admin/</code>	Admin reference-data management (12 sub-forms)
<code>TravelReporting/SetUserAccess.cfm</code>	Role-based access control logic
<code>TRRE-Oracle/TABLE.sql</code>	Full Oracle DDL — source of truth for the domain model
<code>TRRE-Oracle/VIEW.sql</code>	Cross-schema views (staff, org structure, projects)

Target State

Dimension	Target
Backend	.NET 8 minimal API (<code>TravelReporting.Api</code>)
Frontend	React + Vite or Blazor Server (<code>TravelReporting.Web</code>) — choose one lane per team
ORM / Data access	Entity Framework Core 8 with parameterised queries

Dimension	Target
Database	Azure SQL (migrated from Oracle TRRE)
Authentication	Microsoft Entra ID (Azure AD) — replacing NT integrated auth
Hosting	Azure App Service
CI/CD	Azure DevOps (build → test → security scan → deploy)
Tests	xUnit with ≥ 70 % coverage on migrated service layer

Agenda

Day	Phase	Topic	Time
Day 2	1	Analyse & Refactor Legacy Code	60 min
Day 2	2	Frontend Migration	60 min
Day 2	3	Backend Migration	60 min
Day 2	4	Design Target Architecture Patterns	60 min
Day 2	5	CI/CD Pipeline Design & Automation	60 min
Day 3	6	Unit Test Generation for Migrated Code	60 min
Day 3	7	Security Review (OWASP Top 10) + API Modernisation	60 min
Day 3	8	Architecture Review & Pipeline Gates	60 min

The Phases

Phase 1 — Analyse & Refactor Legacy Code

Ask mode

The first step in any modernisation is honest assessment. ColdFusion mixes markup, business logic, and data access on the same page — which means every `.cfm` file is doing several jobs at once. Before writing a line of .NET, use Copilot to map what exists, name the anti-patterns, and produce a refactor plan your team can execute systematically.

Your Challenge

Use GitHub Copilot in Ask mode to analyse the ColdFusion source and produce a written assessment of the functional modules, the anti-patterns, the SQL injection surfaces, and a prioritised refactor plan for the migration.

Things to Explore/Do

- Open `TravelReporting/application.cfm` and ask Copilot: "Explain what this ColdFusion application.cfm does, what security risks it contains, and what the .NET equivalent would look like."
- Open `TravelReporting/Travel/Edit.cfm` — ask Copilot to identify every `<cfquery>` block and explain the SQL injection risk in each one.
- Open `TravelReporting/Travel/Save.cfm` — ask Copilot to map every form field to the Oracle column it writes to, so you can build your EF Core entity later.
- Open `TRRE-Oracle/TABLE.sql` — ask Copilot to generate a C# class hierarchy (domain entities) from the Oracle DDL for the `TRRE.TRAVEL` table.
- Open `TravelReporting/Menu.cfm` — ask Copilot to list every functional area and propose the equivalent .NET 8 minimal API endpoints.
- Ask Copilot: "What is the strangler fig pattern and how would I apply it to migrate TravelReporting from ColdFusion to .NET 8 incrementally?"
- Ask Copilot to review `TravelReporting/SetUserAccess.cfm` and describe the role-based access model so it can be replicated with ASP.NET Core authorisation policies.
- Use the Awesome Copilot legacy-analysis prompt template if available, or craft your own `.instructions.md` scoped to `TravelReporting/**/*.cfm`.

What Done Looks Like

- ☐ You have a written list of the functional modules (Travel CRUD, Search, Reports, Graphs, Admin reference data)
- ☐ You have identified at least 5 SQL injection surfaces in the ColdFusion code

- ☐ You have a draft C# domain model for the `TRAVEL` table from `TRRE-Oracle/TABLE.sql`
- ☐ You have a proposed .NET 8 API endpoint list mapping to the ColdFusion page-per-action pattern
- ☐ You have a prioritised migration order (critical path vs. low-risk items)

🌟 **Stretch Goal** — Create a `.github/copilot-instructions.md` file in your working branch that tells Copilot about the ColdFusion-to-.NET migration context, so every future prompt automatically inherits that knowledge.

🛑 GIT CHECKPOINT

```
git add . && git commit -m "phase-1: legacy analysis complete - domain model"
```

Phase 2 — Frontend Migration

📋 Plan + 🤖 Agent

The TravelReporting UI is a 2002-era HTML frameset with inline styles and no JavaScript framework. Modern users expect responsive, accessible interfaces. Your team will choose one frontend lane — React + Vite for a full SPA experience, or Blazor Server for a .NET-integrated server-rendered approach — and migrate the core Travel proposal form (`TravelReporting/Travel/Edit.cfm`) as a proof of concept.

Your Challenge

Scaffold your chosen frontend project and migrate the Travel proposal form from ColdFusion to your target UI framework.

Things to Explore/Do

React + Vite lane:

- In Plan mode, ask Copilot to design the component tree for the Travel proposal form, based on the 60+ fields in `TravelReporting/Travel/Edit.cfm`.
- Use Agent mode: "Scaffold a React + Vite project called `TravelReporting.Web`. Create a `TravelForm` component with fields mapped from `TravelReporting/Travel/Edit.cfm`. Use React

Hook Form for validation and Axios for API calls."

- Replace the frameset navigation (`TravelReporting/Menu.cfm`) with a React sidebar component with routes matching the functional areas.

Blazor Server lane:

- In Plan mode, ask Copilot to design the Razor component structure for the Travel proposal form.
- Use Agent mode: *"Scaffold a Blazor Server project called TravelReporting.Web targeting .NET 8. Create a TravelEdit.razor component with fields mapped from TravelReporting/Travel/Edit.cfm. Use FluentValidation for model validation."*
- Replace the frameset navigation with a Blazor `NavMenu` component mirroring `TravelReporting/Menu.cfm` .

Both lanes:

- Ask Copilot to migrate the CSS from `TravelReporting/appstyle.css` and `TravelReporting/report.css` to your chosen framework's styling approach.
- Ask Copilot: *"What accessibility issues does the original frameset layout in TravelReporting/Default.cfm have, and how does the migrated layout address them?"*
- Use `@azure-prepare` to provision an Azure App Service slot for the frontend during development.

What Done Looks Like

- ☐ A new frontend project (`TravelReporting.Web`) exists in the workspace
- ☐ The Travel proposal form renders with fields matching `TravelReporting/Travel/Edit.cfm`
- ☐ Client-side validation is in place for required fields
- ☐ The navigation structure mirrors `TravelReporting/Menu.cfm`
- ☐ The frameset layout is replaced with a single-page or component-routed layout

☀ **Stretch Goal** — Implement the Search form (`TravelReporting/Search/Select.cfm`) as a second migrated screen, connected to the backend API from Phase 3.

Phase 3 — Backend Migration

Agent mode

The ColdFusion backend is a flat collection of `.cfm` files with inline SQL — no service layer, no dependency injection, no repository pattern. You will scaffold a .NET 8 minimal API, define EF Core entities from the Oracle schema, and implement the core Travel CRUD endpoints, eliminating the SQL injection risk in the process.

Your Challenge

Scaffold a .NET 8 minimal API project with EF Core 8 targeting Azure SQL. Implement at minimum: list travels, get travel by ID, create travel, and update travel status.

Things to Explore/Do

- Use Agent mode: "Scaffold a .NET 8 minimal API project called `TravelReporting.Api`. Add Entity Framework Core 8 with SQL Server provider. Create a `Travel` entity based on `TRRE-Oracle/TABLE.sql` with all non-deprecated columns. Create a `TravelDbContext`."
- Ask Copilot to generate EF Core entities for `ARRANGER_GROUP` , `APPROVER_USER` , `FLEX_COST_CODE` , and the reference data tables from `TRRE-Oracle/TABLE.sql` .
- Implement the following endpoints:

```
GET /health
GET /travels
GET /travels/{id}
POST /travels
PUT /travels/{id}/status
GET /travels/search?departFrom=&destination=&status=
```

- Ask Copilot: "Convert the inline SQL in `TravelReporting/Travel/Save.cfm` to a parameterised EF Core `SaveChangesAsync` call in C#." — this is the key security fix.
- Replace the NT auth from `TravelReporting/application.cfm` with ASP.NET Core middleware accepting a JWT from Entra ID. Use `[Authorize]` with a placeholder scheme if full Entra setup is out of scope for the session.
- Use `@azure-prepare` to scaffold the Azure SQL connection string and Key Vault reference for the database connection.
- For the cross-schema views (`V_STAFF_CUR` , `V_COSM_STRUCTURE` from `TRRE-Oracle/VIEW.sql`), seed representative data rather than live Oracle integration.

What Done Looks Like

- ☐ `TravelReporting.Api` builds with `dotnet build`
- ☐ `GET /health` returns 200
- ☐ `GET /travels` returns a seeded list of travel records
- ☐ `POST /travels` creates a travel record with parameterised EF Core (no raw SQL)
- ☐ `PUT /travels/{id}/status` updates the approval status
- ☐ No raw SQL strings appear anywhere in the .NET codebase
- ☐ Connection string is read from environment variable or Key Vault reference, not hardcoded

🌟 **Stretch Goal** — Implement `GET /travels/search` with all filter parameters from `TravelReporting/Search/Select.cfm`, using EF Core LINQ expressions rather than dynamic SQL concatenation.

|| GIT CHECKPOINT

```
git add . && git commit -m "phase-3: .NET 8 minimal API scaffolded with EF C
```

Phase 4 — Design Target Architecture Patterns

📋 Plan mode

With a working skeleton in place, it is time to deliberate on architecture. The ColdFusion application is a flat monolith — every concern lives in the same file. .NET gives you the tools to impose structure. This phase is about making a conscious, defensible choice about how to organise the code for maintainability, testability, and future growth.

Your Challenge

Choose and document the target architecture pattern for `TravelReporting.Api` and justify it against the size and complexity of the discovered codebase.

Things to Explore/Do

- In Plan mode, ask Copilot: "Compare Clean Architecture, Vertical Slice Architecture, and a simple layered monolith for `TravelReporting.Api`. The application has approximately 40

ColdFusion pages covering Travel CRUD, Search, Reports, Graphs, and Admin reference data. Which pattern is most appropriate and why?"

- Ask Copilot to generate a folder structure diagram for your chosen pattern applied to `TravelReporting.Api`.
- Consider the Admin sub-modules in `TravelReporting/Admin/` (12 forms for reference data management) — ask Copilot whether these belong in the same API or a separate admin service.
- Ask Copilot: "How should the cross-schema Oracle views `V_STAFF_CUR` and `V_COSM_STRUCTURE` be handled in the target architecture — as seeded data, external service calls, or cached reference data?"
- Use Plan mode to design the Entra ID authentication flow that replaces the `CGI.AUTH_USER` pattern in `TravelReporting/application.cfm`. The new flow must extract user identity from JWT claims and map it to the roles defined in `TravelReporting/SetUserAccess.cfm`.
- Ask Copilot to produce an Architecture Decision Record (ADR) for the chosen pattern — store it as `docs/adr-001-architecture-pattern.md`.

What Done Looks Like

- ☐ Your team has agreed on and documented an architecture pattern (Clean Architecture / Vertical Slices / Layered)
- ☐ `TravelReporting.Api` folder structure reflects the chosen pattern
- ☐ An ADR exists capturing the decision and rationale
- ☐ The authentication strategy (Entra ID replacement for NT auth) is documented
- ☐ The approach to Oracle cross-schema views is decided and documented

🌟 **Stretch Goal** — Apply the Vertical Slice pattern to one complete slice end-to-end: `CreateTravel` (command, handler, validator, endpoint, EF Core entity, test) using the CQRS pattern with MediatR.

Phase 5 — CI/CD Pipeline Design & Automation

Agent mode

The ColdFusion application has no CI/CD pipeline — it was presumably deployed by copying files to a server. For the modernised application, automated pipelines are non-negotiable. This

phase uses Copilot to generate Azure DevOps pipelines that build, test, scan, and deploy `TravelReporting.Api` and `TravelReporting.Web`.

Your Challenge

Generate Azure DevOps pipeline YAML files for the .NET 8 API and your chosen frontend, covering build, test, code analysis, and deployment to Azure App Service.

Things to Explore/Do

- Use Agent mode: "Generate an Azure DevOps pipeline YAML for `TravelReporting.Api` targeting .NET 8. Include stages: restore, build, unit test with coverage threshold 70%, publish artifact, deploy to Azure App Service using a service connection."
- Use Agent mode to generate a separate pipeline for `TravelReporting.Web` :
 - **React + Vite**: `npm ci → npm run build → publish static artefact → deploy`
 - **Blazor Server**: `dotnet publish → publish artefact → deploy`
- Browse github.com/github/awesome-copilot for Azure DevOps pipeline `.prompt.md` templates and adapt them to the DWER travel application context.
- Add a **pipeline variable group** in Azure DevOps for: `AZURE_SQL_CONNECTION_STRING` , `AZURE_APP_SERVICE_NAME` , `ENTRA_CLIENT_ID` .
- Ask Copilot: "Add a code coverage gate to the Azure DevOps pipeline that fails the build if coverage drops below 70%."
- Use `@azure-deploy` or the Azure MCP Server to validate the App Service deployment slot configuration without leaving VS Code.
- Ask Copilot to generate a `.azuredevops/pull-request-template.md` that requires reviewers to confirm: tests pass, security review done, OWASP checklist completed.

What Done Looks Like

- ☐ `azure-pipelines.yml` exists and covers build and test for `TravelReporting.Api`
- ☐ A pipeline for `TravelReporting.Web` exists
- ☐ A 70 % code coverage gate is configured
- ☐ Deployment to Azure App Service is scripted (not manual)
- ☐ Secrets are in variable groups, not hardcoded in YAML
- ☐ Pipeline syntax is valid (can be demonstrated via dry-run or Azure DevOps import)

🌟 **Stretch Goal** — Add an infrastructure-as-code stage using Bicep to provision the Azure SQL server and App Service, validated with `az deployment what-if` in the pipeline before

any deployment step.

II GIT CHECKPOINT

```
git add . && git commit -m "phase-5: Azure DevOps pipelines added - build, t
```

Phase 6 — Unit Test Generation for Migrated Code

Agent mode

The original TravelReporting system has zero tests. Every change is a leap of faith. This phase uses Copilot to generate a meaningful xUnit test suite for the migrated .NET 8 API, targeting the service layer and API endpoints that replaced the most complex ColdFusion logic.

Your Challenge

Generate xUnit tests for the migrated `TravelReporting.Api` service layer and API endpoints, targeting $\geq 70\%$ code coverage on the service classes.

Things to Explore/Do

- Use Agent mode: "Generate xUnit tests for `TravelReporting.Api` covering: create travel (valid input), create travel (missing `DESTINATION` field), get travel by ID (found), get travel by ID (not found), update travel status (valid transition), update travel status (invalid transition)."
- Ask Copilot to generate tests for the approval-deadline logic derived from the `MAX_APPROVAL_DATE` field in `TravelReporting/Travel/Edit.cfm`.
- Ask Copilot to generate integration tests using `WebApplicationFactory<Program>` for the minimal API endpoints.
- For the search endpoint, ask Copilot: "Generate parameterised xUnit tests for `GET /travels/search` with each filter combination found in `TravelReporting/Search/Select.cfm`."
- Generate a test for the security constraint: "Write an xUnit test that verifies a user without the Approver role cannot call `PUT /travels/status` and receives a 403 response."
- Run `dotnet test --collect:"XPlat Code Coverage"` and review the coverage report.
- Use Agent mode to increase coverage on any gaps identified by the report.

What Done Looks Like

- ☐ A `TravelReporting.Tests` project exists with xUnit
- ☐ At least 10 meaningful test cases are implemented (not just stubs)
- ☐ Code coverage report shows $\geq 70\%$ on the service layer
- ☐ At least one integration test using `WebApplicationFactory` is passing
- ☐ At least one security-boundary test (authorisation check) is passing

☀️ **Stretch Goal** — Use Copilot to generate a Stryker.NET mutation testing configuration and run it to verify test quality, not just coverage percentage.

Phase 7 — Security Review (OWASP Top 10) + API Modernisation

💬 Ask + 🤖 Agent

The original ColdFusion application has critical security vulnerabilities — most notably SQL injection on every data-entry form. The .NET migration is an opportunity to address these systematically. This phase combines an OWASP Top 10 review with API modernisation: OpenAPI documentation, input validation, and authentication hardening.

Your Challenge

Run a security review of `TravelReporting.Api` against the OWASP Top 10, fix the identified issues, and add OpenAPI documentation with input validation.

Things to Explore/Do

SQL Injection (A03:2021 — Injection):

- Open `TravelReporting/Travel/Save.cfm` alongside your migrated EF Core code. Ask Copilot: "Does any part of `TravelReporting.Api` construct SQL strings dynamically? Identify any remaining injection risks."
- Verify that all database operations use EF Core parameterised queries — no `FromSqlRaw` with string interpolation.

Broken Access Control (A01:2021):

- Ask Copilot: "Review `TravelReporting.Api` for broken access control. Can a user retrieve or modify another user's travel records? Add authorisation policies to enforce user-scoped data access, replicating the role logic from `TravelReporting/SetUserAccess.cfm`."

Security Misconfiguration (A05:2021):

- Ask Copilot: *"Scan TravelReporting.Api for security misconfiguration: hardcoded connection strings, CORS wildcard, missing security headers, detailed error messages in responses."*
- Enable `UseHsts()` , `UseXContentTypeOptions()` , and review the CORS policy.

Credential Exposure:

- Verify that no NT usernames, database credentials, or email addresses from `TravelReporting/application.cfm` are reproduced in the .NET codebase or committed to source control.

API Modernisation:

- Add Swagger / OpenAPI (`dotnet add package Swashbuckle.AspNetCore`) — ask Copilot to annotate all endpoints with summary, response codes, and schema.
- Add FluentValidation or `IValidatableObject` for `POST /travels` — replicate the required-field rules visible in `TravelReporting/Travel/Edit.cfm` .
- Enable GitHub Advanced Security / CodeQL on the repository and review any findings.
- Ask Copilot: *"Using the Awesome Copilot security review prompt, perform a full OWASP Top 10 review of TravelReporting.Api and produce a remediation checklist."*

What Done Looks Like

- ☐ No raw SQL strings remain in `TravelReporting.Api`
- ☐ Authorisation policies prevent cross-user data access
- ☐ CORS is configured to named origins only (no wildcard `*`)
- ☐ Security headers are set (`HSTS` , `X-Content-Type-Options` , `X-Frame-Options`)
- ☐ `POST /travels` validates required fields and returns `400` with a problem-details body
- ☐ OpenAPI (`/swagger`) is available and documents all endpoints
- ☐ CodeQL scan shows no critical findings (or all findings are documented with mitigations)

🌟 **Stretch Goal** — Add rate limiting middleware (`app.UseRateLimiter()`) to protect the search endpoint, and document the rate limit policy in the OpenAPI spec.

II GIT CHECKPOINT

```
git add . && git commit -m "phase-7: OWASP review complete - SQL injection f
```

Phase 8 — Architecture Review & Pipeline Gates

Plan + Agent

The final phase brings everything together. Your team presents the modernised solution to the facilitator and any attending stakeholders, demonstrates the Azure DevOps pipeline running end-to-end, and agrees on the quality gates that will govern future changes. This is the delivery milestone.

Your Challenge

Finalise the Azure DevOps pipelines with quality gates, conduct a structured architecture review, and produce the deliverables summary your team will present.

Things to Explore/Do

- Use Plan mode to review the pipeline from Phase 5 and add the following gates:
 - **Build gate**: must compile with no warnings treated as errors
 - **Test gate**: must pass all xUnit tests with $\geq 70\%$ coverage
 - **Security gate**: CodeQL scan must pass with no critical findings
 - **IaC gate**: `az deployment what-if` must succeed with no destructive changes
 - **Manual approval**: require a named approver before production deployment
- Ask Copilot: "Add a dotnet format check to the Azure DevOps pipeline that fails if any files are not formatted correctly."
- Use the Azure MCP Server to inspect the deployed App Service:
`#mcp_azure_list_resources` — verify the App Service and Azure SQL are correctly configured.
- Use `@azure-diagnostics` to check App Service logs for any startup errors after the most recent deployment.
- Conduct a structured architecture review walk-through covering:
 1. Source-to-target stack mapping (ColdFusion → .NET 8)
 2. Security improvements demonstrated (before/after the SQL injection fix)
 3. Test coverage report

4. Pipeline demo (trigger a build and show it passing all gates)
 5. Outstanding risks and known gaps
- Ask Copilot to generate `docs/architecture-review-minutes.md` summarising the decisions made across all 8 phases.

What Done Looks Like

- ☐ All five pipeline gates are configured and passing
- ☐ A successful end-to-end pipeline run can be demonstrated
- ☐ The deployed application responds on Azure App Service
- ☐ Architecture review minutes are written and committed
- ☐ At least two security improvements are demonstrated with before/after evidence
- ☐ The team can articulate what remains for a production-ready state

🌟 **Stretch Goal** — Create an Azure DevOps Environment with a protected `production` environment requiring two approvers, a 30-minute wait timer, and deployment branch restrictions to `main` only.

|| GIT CHECKPOINT

```
git add . && git commit -m "phase-8: architecture review complete - pipeline"
```

GHCP Prompting Cheat Sheet

💬 Ask Mode — Understanding & Reviewing

```
Explain what TravelReporting/application.cfm does. List every security risk contains, map each risk to an OWASP Top 10 category, and describe the .NET 8 equivalent for each concern (auth middleware, parameterised queries, HSTS, e
```

```
Open TravelReporting/Travel/Edit.cfm. List every <cfquery> block, identify w each one is vulnerable to SQL injection, and explain what an attacker could
```

each vulnerable query.

@azure-diagnostics Check the startup logs for the TravelReporting.Api App Service and identify any configuration errors or missing environment variables.

Plan Mode — Designing & Architecting

I am migrating TravelReporting from ColdFusion to .NET 8 minimal API. The so application has ~40 ColdFusion pages: Travel CRUD (TravelReporting/Travel/), (TravelReporting/Search/), Reports (TravelReporting/Reports/), Graphs (TravelReporting/Graphs/), and Admin reference data (TravelReporting/Admin/)

Design a Vertical Slice Architecture for TravelReporting.Api with a folder s a list of slices (one per major operation), and the classes each slice should

Design the Entra ID authentication flow that replaces Windows NT integrated (CGI.AUTH_USER) from TravelReporting/application.cfm. The new flow must:

1. Accept a Bearer JWT from Entra ID
2. Extract the user's identity (UPN) from the token claims
3. Map it to a role (Staff / Arranger / Approver / Admin) using the logic in TravelReporting/SetUserAccess.cfm

Show the ASP.NET Core middleware chain and the claims-mapping code.

#mcp_azure_list_resources List all resources in my Azure subscription tagged Environment=hackathon. Verify the App Service and Azure SQL Server are provi and suggest any missing resources needed for TravelReporting.Api.

Agent Mode — Generating & Implementing

Scaffold a .NET 8 minimal API project called `TravelReporting.Api` in the current workspace. Add EF Core 8 with SQL Server provider. Generate a `Travel` entity with all columns from `TRRE-Oracle/TABLE.sql` (`TRRE.TRAVEL` table). Generate a `Travel` with a `DbSet<Travel>`. Generate a `TravelRepository` with methods: `GetAllAsync`, `GetByIdAsync`, `CreateAsync`, `UpdateStatusAsync`. Wire up dependency injection in

Generate xUnit tests for `TravelReporting.Api` covering all happy-path and error scenarios for `POST /travels`. Include: valid travel creation, missing required (`DESTINATION`), invalid status transition, and unauthorised access (no Bearer token). Use `WebApplicationFactory<Program>` for integration tests and `Moq` for unit tests.

Generate an Azure DevOps pipeline YAML (`azure-pipelines.yml`) for `TravelReporting` targeting .NET 8. Include stages: Restore, Build (treat warnings as errors), (xUnit with XPlat Code Coverage, threshold 70%), Security (dotnet format checker), Publish, and Deploy to Azure App Service using service connection `$(AZURE_SERVICE_CONNECTION)`. Read secrets from variable group `TravelReporting`.

Final Deliverables Checklist

- ☐ **Refactor report** — Written assessment of the ColdFusion legacy code: functional modules, anti-patterns, SQL injection surfaces, and prioritised migration plan
- ☐ **Migrated frontend** — `TravelReporting.Web` project with Travel proposal form and navigation migrated to React + Vite or Blazor Server
- ☐ **Migrated backend** — `TravelReporting.Api` .NET 8 minimal API with EF Core, Travel CRUD endpoints, and no raw SQL
- ☐ **Target architecture document** — ADR documenting the chosen architecture pattern (Clean / Vertical Slices / Layered) and authentication strategy
- ☐ **Azure DevOps CI/CD pipeline** — `azure-pipelines.yml` covering build, test, coverage gate, security scan, and App Service deployment
- ☐ **Unit tests with coverage report** — xUnit test suite with $\geq 70\%$ service-layer coverage and a coverage report artefact published by the pipeline

7. ☐ **OWASP security review** — Documented review against OWASP Top 10, evidence of SQL injection fix, access control policies applied, CodeQL findings addressed
 8. ☐ **Modernised API spec + architecture review minutes** — OpenAPI (/swagger) accessible on the deployed App Service; docs/architecture-review-minutes.md committed to the repository
-

Tips for Success

1. **Ask before you generate.** Use Ask mode to understand the ColdFusion code before switching to Agent mode to replace it. Agent-generated code that misunderstands the domain creates more work than it saves.
2. **Prefer Ask → Plan → Agent.** Each mode builds on the last. Skipping straight to Agent on complex problems often produces code that needs significant rework.
3. **Review every Agent output.** Agent mode is fast, but it does not know your organisation's security requirements or the nuances of the DWER travel approval workflow. Always read generated code before committing it.
4. **Commit often.** Use the git checkpoints after each phase. Small, labelled commits make it easy to show what Copilot helped with during the presentation — and easy to recover if a generated change needs to be reverted.
5. **Use MCPs over manual portal clicks.** The Azure MCP Server and @azure-* skills let you inspect and configure Azure resources without leaving VS Code. This is faster and creates an audit trail of what was done.
6. **Stub external dependencies.** The Oracle cross-schema views (V_STAFF_CUR , V_COSM_STRUCTURE) connect to COMA and COSM schemas that will not be available in the hackathon environment. Seed representative data rather than spending time on Oracle connectivity.
7. **Partial completion is fine.** The goal is to demonstrate the approach and the Copilot workflow — not to deliver a production system in two days. A working Phase 1–5 with solid documentation is a better outcome than a rushed Phase 8 with no tests.
8. **Capture your prompts.** When a Copilot prompt produces an excellent result, save it to a .prompt.md file in your repository. These reusable prompts are a deliverable in their own right and demonstrate prompt engineering maturity to stakeholders.